Home
Library
Profile
Stories
Stats
Writing
Your stories
Drafts
Unpublished
Published
Unlisted
Collaborations
Submissions
Manage all your stories in your content list
Documentation

Object Detection Part-3: One Stage Detectors: YOLO

Rajesh Katta
Follow
Jul 12, 2021 · Jul 12, 2021

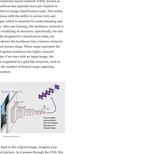
Open in app
Sign up
Sign in
Write
Notifications
More options

Object Detection is a computer vision task that involves identifying and locating objects within an image. This means identifying each object in the image and determining its precise location and boundaries. Common objects that need to be detected include cars, people, and animals. The goal is to find all instances of these objects in the image. Once an object is recognized, its location in the image is typically represented by a bounding box, which is a rectangle that tightly encloses the object. Hence objects in various shapes and sizes, a rectangle might not always fit them perfectly. For example, a bounding box around a person might include some background space or cut off parts of the person. Image Segmentation addresses this limitation by identifying the exact pixels that belong to an object. This allows for more precise object shapes to be detected and localized.

In object detection, there are two main approaches: one-stage and two-stage detectors. One-stage detectors are a class of algorithms that directly predict object bounding boxes and their corresponding class labels in a single pass through the network. They do not rely on separate region proposal steps and are designed to directly predict object locations and classifications from the input image. One-stage detectors are often faster and simpler than two-stage detectors, making them more suitable for real-time applications. Though they lagged, performance wise they've poured significantly over two-stage detectors.

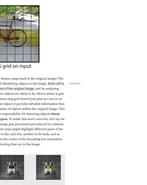
Workflow:

In object detection using a one-stage approach, the workflow begins with inputting the image into a convolutional neural network (CNN), known as the backbone network. The backbone has typically been pre-trained on a large dataset for image classification tasks. As the image passes through the CNN, initial feature maps capture low-level features such as edges and textures. As the network progresses, these feature maps become increasingly rich in high-level semantic information, which is essential for understanding and detecting objects in the image. The feature maps from the backbone network are adapted for object detection by modifying its structure. Specifically, the last few layers, which were originally designed for classification tasks, are removed. The adjustment transforms the backbone into a feature extractor that generates a series of stacked feature maps. These maps represent the image at various levels of abstraction, capturing both spatial and semantic (feature) correlation. For example, if we start with an input image, the resulting feature maps might be organized in a grid like structure, such as a 5 x 5 grid, where each cell represents a portion of feature maps capturing different aspects of the image content.



Feature extraction

To connect these feature maps back to the original image, imagine your image initially divided into a 5 x 5 grid. This grid serves as a framework for dividing pixels combined into a smaller, more abstract version, but can that still retains essential information about the image's content. For instance, if the original image is 250x250 pixels, the resulting feature map might be something like a 5X grid. Each cell in the feature map grid corresponds to a specific region in the original image. For example, if the feature map has three dimensions 40 regions, and each 1x1 cell in the feature map corresponds to a 5x5 region in the original image. The feature map effectively captures information about 25 pixels in the original image, values (channels), each representing different aspects of the image's content in that region. For example, a 1x1x512 feature map means there are 512 different features being tracked for each of these 40 regions.



Now, why do we connect these feature maps back to the original image? The significance lies in locating and identifying objects in the image. Each cell in the feature map grid corresponds to a specific region in the original image. Hence cells, we can predict where objects are likely to be. Hence it's quite interesting. Each cell in the feature map grid doesn't just give us a yes or no answer for object presence, it provides a wealth of information that helps us pinpoint the exact location of objects within the original image. This is done by assigning each cell a responsibility for detecting objects whose midpoints fall within its boundaries. The cell then predicts bounding boxes relative to its grid. For example, if a cell in the feature map grid detected a person in its feature map, different cells in the feature map might highlight different parts of the car: one cell might correspond to the car's tire, another to its body, and so on. The cell region that contains the center of the bounding box annotation of that car is responsible for detecting that car in the image.

Top-left grid cell

Top-right grid cell

Bottom-left grid cell

Bottom-right grid cell

Top-left grid cell

Top-right grid cell

Bottom-left grid cell

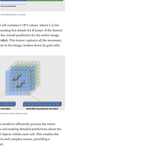
Bottom-right grid cell

Where we predict bounding boxes for objects, we look at these grid cells and decide which ones are responsible for each object. For example, if a cell in the feature map has high activation, stemming it strongly indicates the presence of a car, the cell might predict bounding boxes that cover the car. Bounding box refines its original image based on the information from the particular cell. To accomplish this, we use convolutional filters designed to predict offsets and confidence scores. This involves learning a set of weights that analyze 3D feature maps produced by the backbone network. These filters slide across the feature maps, performing convolutions to predict the classes are likely to be present in each grid cell, and how to describe the bounding box for possible objects within each grid cell.

So, each grid cell in the feature map predicts information that helps identify and locate objects within that cell. For each cell, this prediction includes class probabilities, bounding box coordinates, and confidence scores.

The prediction for each grid cell combines several elements:

1. The likelihood that the cell contains an object, known as the confidence score.
2. The class probabilities, indicating which class the object belongs to.
3. The coordinates and dimensions of the bounding box that locates the object.



one bounding box

Grid cell predicts one bounding box

For each grid cell, if we predict 3 bounding boxes, each bounding box prediction includes five components:

A better place to publish.
Read beyond the ceiling of creativity at Medium.
Sign up
Sign in

x, y coordinates of the center of the bounding box relative to the grid cell, p c confidence of the center of the bounding box relative to the grid cell, w, h width of the bounding box, either normalized by the image width and height or relative to the grid cell dimensions, Confidence score, indicating the likelihood that the bounding box contains an object and how accurate the box is.

If there are C classes, each grid cell also predicts C class probabilities.

one bounding box

Grid cell predicts one bounding box

So, the prediction for each grid cell contains C+5 values: where C is the class probabilities, 5*3 is the bounding boxes for 3 boxes, if the feature map size is 5x5 grid cells, then the overall prediction for the entire image will be a tensor of size **5x5x(C+5)**. This tensor captures all the necessary information for detecting objects in the image, broken down by grid cells.

Feature Extraction

This tensor encounters allows the model to efficiently process the entire image, detecting both objects and making bounding box predictions simultaneously. Thus, it enables real-time object detection capabilities. This makes the model is handle multiple objects and complex scenes, providing a comprehensive detection output.

YOLO: You Only Look Once

The YOLO model (You Only Look Once) is a groundbreaking approach to real-time object detection introduced by Redmon et al. in 2016. Which is unique because it bypasses the region proposal step that many other object detection models rely on, instead directly predicting bounding boxes and class probabilities in a single pass. This end-to-end approach makes YOLO particularly well-suited for applications requiring real-time processing.

YOLO workflow

The YOLO workflow can be broken down into two main steps:

1. **Dividing a CNN:** The process starts by pre-training a Convolutional Neural Network (CNN) on an image classification task. This helps the network learn to extract relevant features from images.
2. **Image Grid and Responsibility:** The input image is divided into an S x S grid. Each grid cell is responsible for detecting objects whose center falls within that cell.
3. **Predictions per Cell:** Each cell predicts:
   - The location of B bounding boxes.
   - A confidence score for each bounding box, indicating the likelihood of containing an object.
   - C class probabilities for object class, conditioned on the existence of an object in the bounding box. The bounding box coordinates are defined by four values (x, y, w, h), where (x, y) are center coordinates of bounding box, this decide where to put the bounding box in the image. (w, h) are the width and height of the box, they define the size of the box. (x, y), (w, h) are normalized by width and height of the image and thus all between 0-1.
   - The confidence score for each bounding box is defined as **the product of the probability that an object exists in the bounding box and the intersection Union (IoU) between the predicted box and the ground truth box.** This score is high if the box is likely to contain an object and if the predicted box overlaps significantly with the true box.
   - If a cell contains an object, it predicts the probability that this object belongs to each of the C classes. This probability is conditioned on the cell **belonging to class C containing object** where C is a specific class. This means that each cell only predicts one set of class probabilities, regardless of the number of bounding boxes.

For an image, the total number of bounding boxes predicted is S*S*B. Each bounding box has 4 location predictions, 1 confidence score, and C class probabilities. Therefore, the total number of predictions for one image is S*S*(B*5+C). This shape represents the final output tensor of the YOLO model.

Network Architecture

The two functions in YOLO is designed to optimize both the bounding box predictions and class probabilities. It consists of two main components: localization loss and classification loss. Both components are computed as the sum of squared errors.

- **Localization Loss:** This part of the loss measures the error between the predicted bounding box coordinates and the ground truth bounding box.
- **Confidence Loss:** This part measures the error in the confidence score, which indicates whether a bounding box contains an object.

YOLO uses two scale parameters, λcoord and λnoobj, to fine-tune the loss. λcoord controls the importance of the bounding box coordinate predictions, while λnoobj down weights the loss contributed by bounding boxes that do not contain objects. This is important because most bounding boxes in the image will be background or doesn't contain objects of interest.

$$L_{loc} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

$$L_{cls} = \sum_{i=0}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2$$

$$L = L_{loc} + L_{cls}$$

Classification Loss:

$$L_{cls} = \sum_{i=0}^{S^2} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2$$

where

$\mathbb{1}_{i}^{obj}$ = 1 if an object appears in cell i, otherwise 0

$\hat{p}_i(c)$ denotes the conditional class probability for class c in cell i.

The classification loss measures the error in the predicted class probabilities. This loss only penalizes the model if there is an object present in the grid cell.

Localization Loss:

The localization loss measures the error in the predicted bounding box coordinates. This loss ensures that the predicted boxes are as close as possible to the ground truth boxes. The localization loss includes two parts: one for the center coordinates (x,y) and one for the dimensions (width,height).

$$L_{loc} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

$$L_{loc} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

where

$\mathbb{1}_{ij}^{obj}$ = 1 if the j th bounding box in cell i is responsible for detecting the object, otherwise 0.

λ_{coord} controls the weight for the loss in the bounding box coordinates.

Confidence Loss:

The confidence loss measures the error in the confidence score, which indicates whether a bounding box contains an object and how accurate the bounding box is. There are two parts to this loss: one for boxes that contain objects and one for boxes that do not contain objects.

$$L_{conf} = \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2$$

where

$\hat{C}_i$ is the confidence score of the box j in cell i.

$\mathbb{1}_{ij}^{obj}$ = 1 if the j th bounding box in cell i is responsible for detecting the object, otherwise 0.

If no object is detected in the box, then the confidence loss is,

$$L_{noobj} = \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2$$

where

$\mathbb{1}_{ij}^{noobj}$ is the complement of $\mathbb{1}_{ij}^{obj}$

$\hat{C}_i$ is the confidence score of the box j in cell i.

λ_{noobj} is weight factor that down weight the loss contribution.

By balancing these three components, YOLO ensures that the model accurately predicts the bounding box location, confidence scores, and class probabilities, leading to effective and efficient object detection.

Conclusion

YOLO's major advantage is its speed. Because it doesn't require a region proposal step and processes the entire image in a single pass through the network, YOLO can perform real-time object detection. This makes it ideal for applications requiring quick decisions, such as autonomous driving and video data analysis. However, YOLO does have some limitations. It can struggle with detecting small objects or objects that are close together, due to the grid cell structure and the limited number of bounding boxes per cell. YOLO makes it powerful tool in the field of object detection.

Deep Learning
Object Detection
Computer Vision
Machine Learning
Tips
Clap
Highlight
Write a response
Write a response...
Cancel
Discard
Response (2)
Most Helpful
Rajesh Katta
Follow
Thank you for your feedback!
Rajesh Katta
Follow
Thank you for your feedback!
Report response
Show all responses
More from Rajesh Katta
Rajesh Katta
Convolutional Neural Network (CNN) Part-1
Rajesh Katta
The Attention Mechanism: A Deep Dive into Transformers
Rajesh Katta
Activation Functions In Depth
Rajesh Katta
WHAT IS MACHINE LEARNING?
Show all from Rajesh Katta
Recommended from Medium
Data Science Collective
Stanford Just Killed Prompt Engineering With 8 Words (And It Worked Better Than Expected)
Jul 19, 2025
Coding Beauty
Stop Using Nested Loops! Learn Data Structures and Algorithms (DSA) Instead
Jun 30, 2025
Jack Cheng
The 5 point subscriptions I actually use to stay sane in a world of noise, AI slop and cognitive overload
Jul 4, 2026
Towards Data Science
Local LLMs That Can Replace Claude Code
Jul 7, 2026
Towards Data Science
10 JSON Files Saved Me Hundreds of Hours Daily for Data Science...
Jul 7, 2026
Towards Data Science
Stop Asking Your Agents To Learn Deep Learning Basics Math Concepts...
Jul 7, 2026
Show more recommendations
Help
Status
About
Careers
Press
Blog
Privacy
Rules
Terms
Text to speech